

# Домашка 02 — Agile Manifesto vs. мій робочий процес

---

**Автор:** Андрій Пономарьов / Claude Code Opus 4.8 **Завдання:** прочитати Agile Manifesto, стисло описати власний робочий процес, показати відмінності від «класичного» agile, запропонувати покращення. Акцент — на організації роботи в команді.

---

## 1. Що каже Agile Manifesto (коротко)

---

**4 цінності** (ліве важливіше за праве):

1. Люди та взаємодія — понад процеси й інструменти
2. Працюючий продукт — понад вичерпну документацію
3. Співпраця із замовником — понад узгодження умов контракту
4. Готовність до змін — понад слідування плану

**12 принципів** зводяться до кількох ідей: рання й часта поставка цінності, вітати зміни навіть пізно, короткі ітерації, щоденна спільна робота бізнесу й розробників, самоорганізовані команди, технічна досконалість, простота («максимізувати обсяг невиконаної роботи») і регулярна рефлексія для покращення.

DevOps (рекомендована стаття Atlassian + домашка 01 про CALMS) — це продовження agile: agile прибрав бар'єри *всередині* команди розробки, DevOps прибирає бар'єр між **dev** і **ops** через автоматизацію, CI/CD і культуру спільної відповідальності. Agile Manifesto (2001) написаний ще до CI/CD — тому «класичний agile» сам по собі CI/CD не вимагає.

---

## 2. Мій поточний робочий процес (3 реальні кейси)

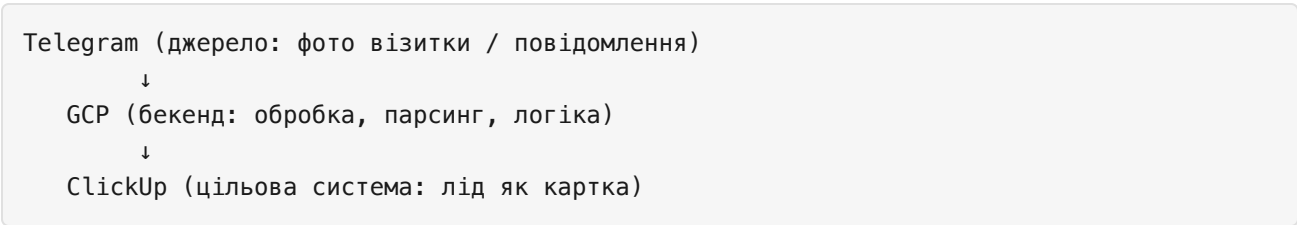
---

### Кейс А — робота з Claude Code (цей курс)

Цикл максимально короткий: **я ставлю задачу природною мовою** → **агент виконує** → **я ревіюю результат** → **коригую**. Немає спринтів, естимейтів, беклогу й церемоній — це безперервний потік (ближче до Kanban/DevOps, ніж до Scrum). «Команда» = одна людина (я — у ролі product owner + reviewer) плюс AI-агент (у ролі розробника). Комунікація асинхронна, текстова, але зворотний зв'язок миттєвий.

### Кейс В — інструмент збору візиток і лідів

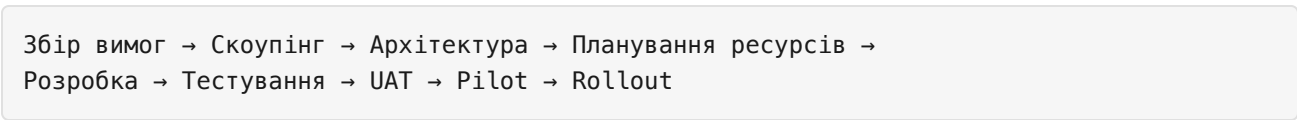
Шлях: від розмитої ідеї «а давайте якось зберігати візитки та ліди» → до робочої архітектури за лічені ітерації:



Тулчейн: **Claude** → **Git** → **GitHub** → **GitHub Actions** — тобто повноцінний CI/CD. Тестування йде *в процесі*, а не окремою фазою наприкінці. Архітектура не була спроектована «згори донизу» — вона **проросла** з потреби (емерджентний дизайн, принципи 10–11 маніфесту).

Кейс С — як це робилося в банку ~20 років тому

Класичний waterfall, послідовні фази без перетину:



CI/CD **немає**. Реліз — рідкісна, велика, ризикована подія. Зворотний зв'язок від замовника по суті лише на UAT/pilot — у самому кінці, коли зміна вже дорога. Команда розбита на «силоси» за фазами (аналітики → архітектори → розробники → QA → ops), кожен «перекидає» роботу далі.

3. Відмінності від «класичного» agile

| Аспект            | Кейс С (банк, waterfall)     | Класичний agile (Scrum)  | Кейси А/В (мій сьогоdnішній)     |
|-------------------|------------------------------|--------------------------|----------------------------------|
| Поставка          | рідко, наприкінці            | кожні 1–4 тижні (спринт) | безперервно, по готовності       |
| Зворотний зв'язок | лише UAT/pilot               | демо в кінці спринта     | у межах хвилин (рев'ю одразу)    |
| Зміни             | дорогі, через change request | вітаються між спринтами  | вітаються будь-коли, в потоці    |
| Документація      | вичерпна, наперед            | помірна                  | мінімальна, код = джерело правди |
| Архітектура       | велике проєктування наперед  | потроху                  | емерджентна (В)                  |

| Аспект  | Кейс С (банк, waterfall) | Класичний agile (Scrum)           | Кейси A/B (мій сьогоднішній) |
|---------|--------------------------|-----------------------------------|------------------------------|
| Команда | силоси за фазами         | крос-функційна, самоорганізована  | людина + AI-агент            |
| CI/CD   | немає                    | не обов'язковий (поза маніфестом) | базовий від першого дня (B)  |

**Ключове спостереження.** Банк (C) суперечить майже всім цінностям маніфесту: процес понад людей, документація понад продукт, план понад зміни. Мій сьогоднішній процес (A/B) реалізує цінності agile й одночасно **виходить за межі маніфесту 2001 року** в бік DevOps: CI/CD, автоматизоване тестування, один безперервний потік замість ітерацій-«коробочок».

Цікавий нюанс щодо **командної організації**: agile передбачає *самоорганізовану крос-функційну команду людей* (принципи 4, 6, 11) і «спілкування віч-на-віч» як найефективніше. У моєму випадку команда переозначена — **1 людина + AI**. «Віч-на-віч» замінено на асинхронний текст, але затримка зворотного зв'язку близька до нуля, тож дух принципу (швидка взаємодія) збережено, навіть якщо буква (face-to-face) — ні.

## 4. Що можна покращити

- Definition of Done наперед.** Перед тим як агент стартує задачу — фіксувати критерії приймання. Зменшує переробку (зараз вони часто проявляються вже під час рев'ю).
- Регулярна рефлексія (принцип 12).** Зараз ретро — стихійне. Варто раз на N задач коротко фіксувати «що сповільнювало» і правити CLAUDE.md / пам'ять агента.
- Закрити петлю DevOps на кейсі B — monitor/operate.** Додати базову спостережуваність (логи помилок парсингу візиток, алерти), щоб після deploy був ще й monitor — а не лише «викотили і забули».
- Версіонувати знання команди.** CLAUDE.md, пам'ять, конвенції — це «онбординг» для майбутніх учасників (людей чи інших агентів). Тримати їх як код.
- Готовність до масштабування команди.** Щойно з'являться ще люди — увімкнути branch protection, обов'язкове code review у GitHub Actions, спільні конвенції. Зараз процес «заточений» під одного оператора.

## 5. Висновок

Класичний agile витіснив waterfall-модель банку (кейс C), прибравши силоси й наблизивши зворотний зв'язок. Мій сьогоднішній процес з AI (кейси A/B) іде ще на крок далі: він **бере цінності agile й додає DevOps-автоматизацію**, а заразом переозначає саме поняття «команда» — від десятка спеціалістів за фазами до зв'язки «людина-замовник/рев'юер + AI-

розробник» із майже миттєвим циклом. Напрямок покращень — не «більше процесу», а замкнути петлю (моніторинг), формалізувати критерії готовності й підготувати процес до повернення *людської* команди.